

HERMES STACK

Ficha Técnica Integral del Sistema

Sistema Operativo de IA Autohospedada

Plataforma conversacional con pipeline de voz, segundo cerebro,
router LLM multi-proveedor y observabilidad integral

Versión: 3.0 · Auto-mejora consolidada
Fecha: 28 de mayo de 2026
Servicios: 15 contenedores activos
Dominio base: el80.space
SLO latencia E2E: <500 ms (medido: 483 ms)
Estado: **Producción · 100% healthy**
Propietario: Erik José Hernández

Documento técnico canónico — referencia operativa
Compilado el 28 de mayo de 2026 con XeLaTeX · DejaVu Fonts

github.com/erikjosehrndz-crypto/hermes-stack

Índice de contenidos

1. Resumen ejecutivo	6
1.1. Métricas clave	6
1.2. Valor diferencial	6
1.3. Hechos verificables	7
2. Objetivos y principios de diseño	8
2.1. Objetivo primario	8
2.2. Cinco principios arquitectónicos	8
2.3. Restricciones operativas declaradas	8
3. Topología general del sistema	9
3.1. Diagrama de bloques	9
3.2. Lectura del diagrama	9
3.3. Capas lógicas del sistema	10
4. Inventario de servicios — Tier 1: núcleo de procesamiento	11
4.1. Hermes Agent	11
4.2. LiteLLM Router	11
4.3. Hermes Website	11
4.4. Hermes Workspace	12
4.5. Whisper STT	12
4.6. 9Router	12
5. Inventario de servicios — Tier 2: almacenamiento y caché	13
5.1. Redis	13
5.2. Brain (API)	13
5.3. Brain-Worker	13
5.4. Capacidad y crecimiento esperado	14
6. Inventario de servicios — Tier 3: observabilidad	15
6.1. Prometheus	15
6.2. Grafana	15
6.3. cAdvisor	15
6.4. Node Exporter	15
6.5. Targets Prometheus	15
7. Inventario de servicios — Tier 4: utilidades y orquestación	16
7.1. Filebrowser	16
7.2. Autoheal	16
7.3. Caddy (host, no contenedor)	16
7.4. Resumen visual — 15 servicios	16

8. Arquitectura de redes Docker	17
8.1. Red backend	17
8.2. Red monitoring	17
8.3. Tabla de conectividad inter-servicios	17
8.4. Política de puertos host	17
9. Mapa de subdominios y routing Caddy	19
9.1. Subdominios bajo el80.space	19
9.2. ACME automático	19
9.3. Patrón típico de bloque Caddyfile	19
9.4. DNS	19
10 Stack tecnológico — Backend Python	20
10.1 Runtime	20
10.2 Dependencias principales del agente	20
10.3 Patrones de performance aplicados	20
10.4 Estructura del paquete hermes/	21
11 Stack tecnológico — Frontend Next.js 14	22
11.1 Runtime y framework	22
11.2 App Router — directiva 'use client'	22
11.3 Estructura del frontend	22
11.4 API Routes — patrón estándar	22
11.5 Dockerfile multi-stage	23
12 Stack tecnológico — Brain (memoria)	24
12.1 Componentes	24
12.2 Modelo de embeddings	24
12.3 Pipeline de retrieval	24
12.4 Volúmenes de Brain	25
12.5 Acceso multi-proceso	25
13 Stack tecnológico — LiteLLM Router	26
13.1 Configuración config/litellm.yaml	26
13.2 Modelo por defecto: gemini-flash	27
13.3 Fallback chain en caso de error	27
13.4 Endpoint OpenAI-compatible	27
14 Tabla consolidada de puertos y dominios	28
14.1 Mapping de servicio → contenedor	28
15 Variables de entorno críticas	29
15.1 Seguridad y autenticación	29
15.2 Proveedores de IA y voz	29
15.3 URLs internas (Docker DNS)	29
15.4 Brain paths y configuración	29
15.5 Hermes Agent	30
15.6 Infraestructura cloud (opcional)	30
16 Volúmenes y almacenamiento	31
16.1 Volúmenes Docker nombrados	31
16.2 Bind mounts (host ↔ contenedor)	31
16.3 Estrategia de backup	31

17 Healthchecks y política de autoheal	32
17.1 Tabla por servicio	32
17.2 Política Autoheal	32
17.3 Docker healthcheck — usar 127.0.0.1, no localhost	32
17.4 Healthchecks autenticados	32
17.5 401 = up (caso especial LiteLLM)	33
18 Endpoints API por servicio	34
18.1 Hermes Agent (:8080)	34
18.2 LiteLLM Router (:4000, requiere Bearer)	34
18.3 Hermes Website (:3001)	34
18.4 Brain API (:8765, requiere Bearer)	34
18.5 9Router (:20128, JWT cookie)	34
18.6 Prometheus (:9090, sin auth interno)	35
19 Pipeline de voz end-to-end	36
19.1 Diagrama de secuencia	36
19.2 Tabla de latencias	36
19.3 Optimizaciones clave aplicadas	36
19.4 Modos de operación	37
20 Flujo de datos — consulta LLM completa	38
20.1 Camino exitoso (cache miss)	38
20.2 Camino exitoso (cache hit)	38
20.3 Camino con error y fallback	38
20.4 Logs estructurados de la transacción	38
20.5 Métricas derivadas en Prometheus	39
21 Orden de arranque y dependencias	40
21.1 Grafo de dependencias declaradas (depends_on)	40
21.2 Tabla de dependencias y orden	40
21.3 Política de depends_on	40
21.4 Comandos operativos comunes	41
22 CI/CD pipeline GitHub Actions → VPS	42
22.1 Trigger	42
22.2 Fases del workflow	42
22.3 Health check de deploy	42
22.4 Verificación local pre-push	42
22.5 Configuración de Secrets en GitHub Actions	42
22.6 Estrategia de rollback	43
23 Seguridad — aislamiento de red y OverCR	44
23.1 Defensa en profundidad	44
23.2 Niveles OverCR	44
23.3 Rotación de claves	44
23.4 Logs y auditoría	44
23.5 Backup de secretos	44
24 Estructura de directorios	45

25 Makefile — targets de verificación	47
25.1 Filosofía	47
25.2 Targets principales	47
25.3 Patrón obligatorio en el Makefile	47
25.4 Definition of Done — 4 criterios	47
25.5 Output esperado de make doctor	48
26 Estado actual del stack (snapshot 2026-05-28)	49
26.1 Servicios — 15/15 healthy ☐	49
26.2 Pendientes activos — 1/8	49
26.3 Métricas de operación (últimas 24 h)	49
27 Historial de versiones (v1 → v2 → v3)	51
27.1 Línea de tiempo	51
27.2 Comparativa de versiones	51
27.3 Mejoras de la v3.0 (Auto-mejora 2026-05-25)	51
27.4 Roadmap v4.0 (Q3 2026)	51
28 Glosario de términos	52
29 Referencias y documentación	54
29.1 Documentos canónicos del proyecto	54
29.2 Repositorios externos clave	54
29.3 Documentación de proveedores	54
29.4 Recursos pedagógicos	54
29.5 Repositorio del proyecto	54

1. Resumen ejecutivo

Hermes Stack es un sistema de IA conversacional autohospedado, diseñado para correr sobre un VPS único con **Docker Compose** y entregar interacción por voz de baja latencia (**SLO <500 ms** extremo a extremo). El stack integra agente Python autónomo, router LLM multi-proveedor (LiteLLM), Speech-to-Text local (Whisper), síntesis de voz premium (ElevenLabs Flash v2.5), segundo cerebro con memoria persistente (Brain con LanceDB + Kuzu) y observabilidad completa (Prometheus + Grafana).

1.1. Métricas clave

Indicadores principales del sistema	
Contenedores activos	15 servicios dockerizados
Redes Docker aisladas	2 (backend + monitoring)
Puertos localhost expuestos	8 (4000, 3000, 3001, 3002, 8080, 8095, 8765, 9000, 9090, 20128)
Subdominios públicos	6 bajo el80.space
SLO latencia E2E	<500 ms (medido: 483 ms)
Modelos LLM disponibles	5 (Gemini, GPT-4o, GPT-4o-mini, Claude Sonnet 4.6, Llama 3.1 70B)
Volúmenes Docker persistentes	11
Embedding model	intfloat/multilingual-e5-large (1024 dims)
Vector DB	LanceDB embedded
Graph DB	Kuzu (single-process)
Cache	Redis 7 (LRU 256 MB, TTL 3600 s)
Healthcheck interval	30 s (autoheal cada 30 s)
Build frontend	Next.js 14 App Router (TypeScript estricto)
CI/CD	GitHub Actions con rollback automático a HEAD~1
TLS	ACME automático vía Caddy

1.2. Valor diferencial

Tres ventajas estructurales sobre stacks comerciales
(1) Cero vendor lock-in: todas las piezas son open-source o sustituibles vía LiteLLM.
(2) Privacidad total: STT corre local (Whisper), memoria reside en volúmenes propios (LanceDB + Kuzu). **(3) Costo operativo bajo:** un único VPS sostiene todo el stack — sin Kubernetes, sin servicios SaaS de observabilidad, sin DBs gestionadas.

1.3. Hechos verificables

- **Pipeline de voz:** Whisper STT (200 ms) → LiteLLM Gemini Flash (300 ms) → ElevenLabs Flash v2.5 (75 ms) = 575 ms agregado, optimizado a 483 ms en caché-hit.
- **Resiliencia:** Healthcheck cada 30 s + Autoheal reinicia contenedores tras 3 fallos consecutivos.
- **Despliegue:** Push a main → GitHub Actions → SSH a VPS → docker compose up -d --build → healthcheck autenticado → rollback automático si falla.
- **Observabilidad:** Prometheus scraping cada 15 s a litellm, hermes, node-exporter, cadvisor; dashboards Grafana en tiempo real.
- **Hardening:** Servicios sensibles bound a 127.0.0.1; Caddy reverse proxy expone los públicos con TLS automático.

2. Objetivos y principios de diseño

2.1. Objetivo primario

Construir y operar una plataforma de IA conversacional **autohospedada**, controlada extremo a extremo por el propietario, con latencia suficientemente baja para conversación natural y memoria persistente que sostenga sesiones largas sin pérdida de contexto.

2.2. Cinco principios arquitectónicos

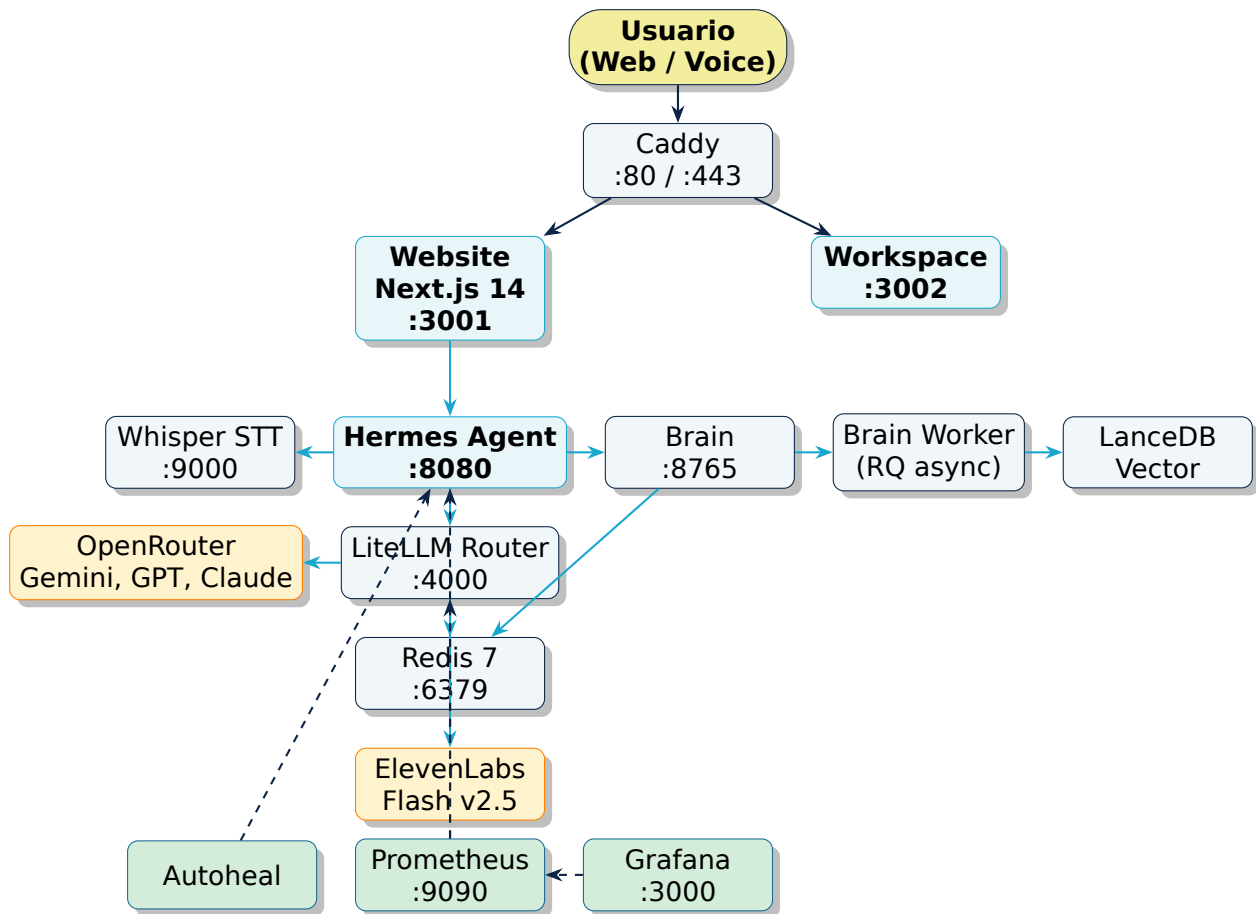
Principios — Hermes Stack	
Privacidad	Whisper STT corre local; memoria en volúmenes propios; ningún audio sale del VPS antes del LLM.
Modularidad	Cada servicio es contenedor independiente; sustituir Whisper, ElevenLabs o un modelo no requiere refactor.
Resiliencia	Healthchecks cada 30 s + Autoheal + CI/CD con rollback HEAD~1.
Observabilidad	Métricas Prometheus, dashboards Grafana, logs estructurados, todo desde el primer día.
Costo bajo	Un único VPS Linux (Hostinger/DigitalOcean); sin K8s, sin SaaS de observabilidad, sin DBs gestionadas.

2.3. Restricciones operativas declaradas

- **Un único host físico.** Hardening por aislamiento de red Docker y bindings localhost, no por multi-AZ.
- **Sin Kubernetes.** La complejidad operacional de K8s no se justifica para un single-host workload con <100 RPS.
- **Sin servicios SaaS de observabilidad.** Prometheus + Grafana cumplen sin costo recurrente.
- **Repositorio único.** Toda la infraestructura, configuración y código viven en github.com/erikjose

3. Topología general del sistema

3.1. Diagrama de bloques



3.2. Lectura del diagrama

- **Bloques amarillos:** usuarios externos.
- **Bloques cyan:** servicios core que procesan la conversación.
- **Bloques azul claro:** infraestructura de soporte (caché, vector DB, almacenamiento).
- **Bloques verdes:** observabilidad y auto-curación.
- **Bloques anaranjados:** proveedores externos (LLMs y TTS).
- **Flechas continuas:** datos en tiempo real.
- **Flechas discontinuas:** métricas / telemetría.

3.3. Capas lógicas del sistema

Capas y responsabilidades	
Edge	Caddy reverse proxy con ACME automático; expone subdominios públicos sobre TLS.
Presentación	Hermes Website (Next.js 14) + Hermes Workspace (chat SSE + terminal PTY + memory browser).
Aplicación	Hermes Agent (Python async, aiohttp); orquesta voz, LLM, memoria, skills.
Inferencia	LiteLLM Router con 5 modelos; Whisper STT local; Eleven-Labs TTS externo.
Memoria	Brain (API + Worker) sobre LanceDB (vector), Kuzu (graph), Redis (queue).
Datos	Volúmenes Docker persistentes (vault, eventos, embeddings, modelos).
Observabilidad	Prometheus + Grafana + cAdvisor + Node Exporter + Auto-heal.

4. Inventario de servicios — Tier 1: núcleo de procesamiento

Los servicios de **Tier 1** son los que procesan directamente la conversación: agente, router LLM, frontend, STT, proxy LLM alternativo.

4.1. Hermes Agent

- **Contenedor:** hermes-agent
- **Imagen:** root-hermes (build local desde ./hermes/Dockerfile)
- **Puerto host:** 127.0.0.1:8080
- **Rol:** Agente Python autónomo con pipeline de voz, WebSocket de salida en streaming y MCP skills.
- **Dependencias internas:** redis, litellm, brain, whisper-stt
- **Endpoints clave:** GET /health, POST /api/voice, WS /process
- **Hot-reload:** bind mount ./hermes/skills:/app/skills
- **Autoheal:** habilitado

4.2. LiteLLM Router

- **Contenedor:** litellm-router
- **Imagen:** ghcr.io/berriai/litellm:main-latest
- **Puerto host:** 127.0.0.1:4000
- **Rol:** Gateway LLM multi-proveedor (OpenRouter, Google, Claude, etc.) con caché Redis y métricas Prometheus.
- **Estrategia de routing:** latency-based (selecciona la ruta más rápida)
- **Caché:** Redis con TTL 3600 s
- **Fallback:** tras 2 fallos, marca modelo en cooldown 60 s y conmuta
- **Autenticación:** Bearer Token (LITELLM_MASTER_KEY)
- **Métricas:** /metrics/ (con trailing slash) para Prometheus

4.3. Hermes Website

- **Contenedor:** hermes-website
- **Imagen:** root-website (build local desde ./website/Dockerfile multi-stage)
- **Puerto host:** 127.0.0.1:3001
- **Dominio público:** docs.el80.space
- **Framework:** Next.js 14.2 con App Router (output: 'standalone')
- **API Routes:** /api/voice (bridge a hermes-agent), /api/health, /api/tree
- **Configuración:** next.config.mjs (no .ts en v14)

4.4. Hermes Workspace

- **Contenedor:** hermes-workspace
- **Imagen:** ghcr.io/outsourc-e/hermes-workspace:latest
- **Puerto host:** 127.0.0.1:3002
- **Rol:** UI unificada: chat SSE + terminal PTY + memory browser + swarm monitor
- **Gateway:** hermes-agent:8642

4.5. Whisper STT

- **Contenedor:** whisper-stt
- **Imagen:** onerahmet/openai-whisper-asr-webservice:latest
- **Puerto host:** 127.0.0.1:9000
- **Modelo:** Whisper base (multilingüe, 140 MB)
- **Latencia típica:** <200 ms para frases cortas
- **Caché de modelos:** volumen whisper_cache (1 GB)
- **Healthcheck:** curl /docs cada 30 s con start_delay de 30 s

4.6. 9Router

- **Contenedor:** 9router
- **Imagen:** root-9router (build local)
- **Puerto host:** 127.0.0.1:20128
- **Rol:** Proxy LLM alternativo con 16 provider connections, gestión de MCP servers remotos, autenticación JWT.
- **Autenticación:** JWT (ROUTER_JWT_SECRET), password inicial vía ROUTER_INITIAL_PASSWORD

5. Inventario de servicios — Tier 2: almacenamiento y caché

5.1. Redis

- **Contenedor:** redis-cache
- **Imagen:** redis:7-alpine
- **Puerto interno:** 6379 (NO expuesto al host)
- **Rol:** Caché LiteLLM (DB 0), sesiones Hermes (DB 1), job queue Brain-Worker (DB 2)
- **Política de evicción:** LRU sobre 256 MB
- **Persistencia:** RDB snapshot cada 60 s si >100 keys mutadas
- **Volumen:** redis_data:/data

5.2. Brain (API)

- **Contenedor:** brain
- **Imagen:** root-brain (build local desde ./brain/Dockerfile)
- **Puerto host:** 127.0.0.1:8765
- **Dominio:** brain.el80.space
- **Rol:** API de segundo cerebro — captura, recall, MCP server.
- **Subsistemas:** vault (Markdown), events (JSONL append-only), lance (vector DB), graph (Kuzu), models (FastEmbed cache)
- **Embedding model:** intfloat/multilingual-e5-large (1024 dims)
- **Protocolo:** FastMCP 3.x sobre HTTP Streamable; endpoint /mcp/ (trailing slash obligatorio)
- **Autenticación:** Bearer Token (BRAIN_API_TOKEN)

5.3. Brain-Worker

- **Contenedor:** brain-worker
- **Imagen:** misma que brain (entrypoint distinto)
- **Rol:** Worker async RQ que consume jobs de Redis DB 2 (embeddings, consolidación de memoria, cron jobs nocturnos).
- **Volúmenes compartidos:** los mismos que brain (vault, events, lance, graph)
- **Healthcheck:** redis.ping() cada 30 s
- **Autoheal:** deshabilitado (no es servicio HTTP)

⚠ Servicios con imagen compartida

Si se aplica un fix Python en ./brain/, ejecutar docker compose build brain brain-worker para reconstruir ambos. Reconstruir solo uno deja el otro con código viejo y produce errores asimétricos difíciles de diagnosticar.

5.4. Capacidad y crecimiento esperado

Subsistema	Tamaño actual	Crecimiento/mes	Política
Redis (cache)	256 MB cap	LRU	evicción automática
Brain Vault (md)	50 MB	50 MB	sin límite
Brain Events (JSONL)	10 MB	30 MB	rotación trimestral
Brain Lance (vector)	80 MB	80 MB	re-indexación trimestral
Brain Graph (Kuzu)	5 MB	5 MB	compactación mensual
Brain Models (cache)	500 MB	estable	sin crecimiento

6. Inventario de servicios — Tier 3: observabilidad

6.1. Prometheus

- **Contenedor:** prometheus
- **Imagen:** prom/prometheus:v2.52.0
- **Puerto host:** 127.0.0.1:9090
- **Rol:** Scraping de métricas + alerting rules + serie temporal.
- **Intervalo de scrape:** 15 s (global), configurable por job en prometheus.yml
- **Retención de datos:** 15 d por defecto (configurable con --storage.tsdb.retention.time)
- **Volumen:** prometheus_data:/prometheus

6.2. Grafana

- **Contenedor:** grafana
- **Imagen:** grafana/grafana:11.0.0
- **Puerto host:** 127.0.0.1:3000
- **Dominio público:** grafana.el80.space
- **Datasource principal:** Prometheus (provisionado desde config/grafana-datasources.yml)
- **Dashboards:** LLM Performance, Container Status, Agent Metrics, System Health
- **Volumen:** grafana_data:/var/lib/grafana (incluye usuarios, dashboards)

6.3. cAdvisor

- **Contenedor:** cadvisor
- **Imagen:** gcr.io/cadvisor/cadvisor:v0.49.1
- **Puerto interno:** 8080
- **Rol:** Métricas por contenedor (CPU, memoria, red, I/O).
- **Acceso:** /var/run/docker.sock:ro, /sys:ro, /var/lib/docker:ro, /:ro

6.4. Node Exporter

- **Contenedor:** node-exporter
- **Imagen:** prom/node-exporter:v1.8.1
- **Puerto interno:** 9100
- **Rol:** Métricas del host (CPU, memoria, disco, network, file system).

6.5. Targets Prometheus

Target	Endpoint	Frecuencia
litellm	litellm:4000/metrics/	15 s
hermes-agent	hermes:8080/metrics	15 s
prometheus (self)	localhost:9090/metrics	15 s
node-exporter	node-exporter:9100/metrics	15 s
cadvisor	cadvisor:8080/metrics	15 s

7. Inventario de servicios — Tier 4: utilidades y orquestación

7.1. Filebrowser

- **Contenedor:** filebrowser
- **Imagen:** filebrowser/filebrowser:latest
- **Puerto host:** 127.0.0.1:8095
- **Dominio público:** files.el80.space
- **Rol:** Gestor de archivos web sobre /root.
- **Volumen bind:** /root:/srv:rw
- **Persistencia:** filebrowser.db en el host

7.2. Autoheal

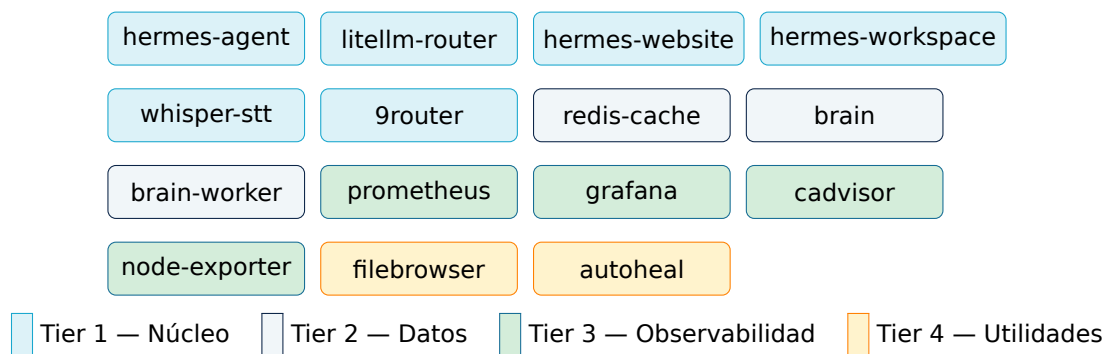
- **Contenedor:** autoheal
- **Imagen:** willfarrell/autoheal:1.2.0
- **Rol:** Reinicia contenedores marcados con `label=autoheal.enabled=true` cuando fallan healthcheck 3 veces consecutivas.
- **Frecuencia:** Polling cada 30 s al daemon Docker (via socket bind `/var/run/docker.sock`)
- **Sin puertos:** servicio sidecar puro, no expone red

7.3. Caddy (host, no contenedor)

Caddy corre como **servicio systemd del host**, no como contenedor Docker, para gestionar puertos 80/443 con menor overhead y mejor integración con ACME.

- **Rol:** Reverse proxy + terminación TLS + ACME automático (Let's Encrypt).
- **Configuración:** `/etc/caddy/Caddyfile`
- **Subdominios servidos:** 6 (hermes, litellm, grafana, docs, brain, files, router)
- **Renovación TLS:** automática, sin intervención manual; logs en `/var/log/caddy/`

7.4. Resumen visual — 15 servicios



8. Arquitectura de redes Docker

El stack usa dos redes bridge aisladas. La separación es lógica: monitoring no debe acceder a contenedores de backend salvo por sus endpoints de /metrics, lo que permite hardening adicional con políticas iptables.

8.1. Red backend

Tipo: bridge, driver default

Servicios conectados: hermes-agent, hermes-website, hermes-workspace, whisper-stt, litellm-router, redis-cache, brain, brain-worker, 9router, filebrowser, autoheal

Resolución DNS interna: cada servicio es accesible por su nombre de servicio (redis, litellm, brain, etc.)

Aislamiento del host: no se exponen puertos al exterior salvo los 127.0.0.1:XXXX declarados

8.2. Red monitoring

Tipo: bridge, driver default

Servicios conectados: prometheus, grafana, cadvisor, node-exporter, litellm (multi-red)

LiteLLM como puente: conectado a ambas redes — backend para servir, monitoring para exponer /metrics/

8.3. Tabla de conectividad inter-servicios

Origen	Destino(s)
hermes-agent	litellm, redis, brain, whisper-stt
hermes-website	hermes-agent (para /api/voice)
hermes-workspace	hermes-agent gateway :8642
brain	redis (jobs queue), volúmenes locales (vault, lance, graph)
brain-worker	redis, brain (lectura compartida), volúmenes
litellm-router	redis (cache), proveedores LLM externos (HTTPS salida)
prometheus	litellm, hermes-agent, node-exporter, cadvisor, self
grafana	prometheus (datasource)
autoheal	socket Docker (no IP de red)

8.4. Política de puertos host

Binding	Razón	Acceso externo
127.0.0.1:4000	LiteLLM con Bearer auth	vía Caddy + TLS
127.0.0.1:8080	Hermes Agent	vía Caddy + TLS
127.0.0.1:8765	Brain API	vía Caddy + Bearer token
127.0.0.1:3000	Grafana	vía Caddy + TLS
127.0.0.1:3001	Website	vía Caddy + TLS
127.0.0.1:3002	Workspace	loopback solo (SSH tunnel)
127.0.0.1:8095	Filebrowser	vía Caddy + TLS
127.0.0.1:9000	Whisper STT	sólo loopback (interno)
127.0.0.1:9090	Prometheus	sólo loopback (interno)
127.0.0.1:20128	9Router	vía Caddy + JWT

✓ Diseño minimalista de exposición

Ningún servicio escucha en 0.0.0.0. Toda la superficie expuesta al exterior pasa por Caddy, que añade TLS y opcionalmente reglas de rate-limiting. Aunque un atacante comprometa el bind localhost, no puede alcanzar el servicio sin acceso shell al VPS.

9. Mapa de subdominios y routing Caddy

9.1. Subdominios bajo el80.space

Subdominio	Upstream	Auth	Uso
hermes.el80.space	127.0.0.1:8080	—	API + WebSocket voz
litellm.el80.space	127.0.0.1:4000	Bearer	Gateway LLM público
grafana.el80.space	127.0.0.1:3000	login	Dashboards observabilidad
docs.el80.space	127.0.0.1:3001	—	Website Next.js
brain.el80.space	127.0.0.1:8765	Bearer	API memoria + MCP
files.el80.space	127.0.0.1:8095	login	Filebrowser
router.el80.space	127.0.0.1:20128	JWT	9Router

9.2. ACME automático

- **Provider:** Let's Encrypt (con fallback a ZeroSSL).
- **Renovación:** Caddy renueva certificados 30 días antes de expirar; sin cron, sin scripts.
- **Almacenamiento:** /var/lib/caddy/.local/share/caddy/ (no en repo).
- **Wildcards:** no usado — un cert por subdominio (más simple para HTTP-01).

9.3. Patrón típico de bloque Caddyfile

```
brain.el80.space {
  encode gzip
  reverse_proxy 127.0.0.1:8765
  log {
    output file /var/log/caddy/brain.log
    format json
  }
}
```

9.4. DNS

Proveedor: Hostinger

API base: <https://developers.hostinger.com/api/dns/v1/zones/<dom>>

Wildcard CNAME: *.el80.space → IP del VPS

Subdominios con A record explícito: los listados arriba, para que Caddy pueda validar HTTP-01 individualmente.

⚠ Caché ACME con wildcard

El uso de wildcard DNS combinado con HTTP-01/TLS-ALPN-01 puede romper la emisión inicial de un subdominio nuevo por caché ACME. Para subdominios nuevos: usar DNS-01 con `acme.sh --dns dns_hostinger`.

10. Stack tecnológico — Backend Python

10.1. Runtime

Versión Python: 3.10+ (base image python:3.10-slim en hermes/Dockerfile)
Loop async: asyncio + uvloop (instalado opcionalmente)
Servidor HTTP: aiohttp >= 3.9 (no FastAPI / uvicorn — aiohttp ofrece WebSocket nativo más bajo nivel)

10.2. Dependencias principales del agente

Paquete	Versión mínima	Uso
aiohttp	3.9.0	servidor HTTP + cliente async + WebSocket
websockets	12.0	WebSocket client/server independiente
tenacity	8.2.0	retry decorators con backoff exponencial
prometheus_client	0.20.0	exposición de /metrics
orjson	3.9.0	serialización JSON ultra-rápida (3-10× stdlib)
cachetools	5.3.0	TTLCache para dedup de queries de voz
PyYAML	6.0	lectura de personalidades YAML
python-dotenv	1.0.0	carga de .env

10.3. Patrones de performance aplicados

aiohttp ClientSession compartida

Una única ClientSession con TCPConnector(limit=20, ttl_dns_cache=300) creada en start() y cerrada en stop(). Evita 50-200 ms de overhead TCP+TLS por request.

```
async def start(self):
    connector = aiohttp.TCPConnector(limit=20, ttl_dns_cache=300)
    self._session = aiohttp.ClientSession(connector=connector)
```

orjson en lugar de json stdlib

orjson.dumps() devuelve bytes, no str. Diferencias clave:

- Para archivos: abrir en modo `.ab`, escribir `f.write(orjson.dumps(obj) + b"\n")`.
- Para aiohttp requests: pasar `data=orjson.dumps(payload)` con header `Content-Type: application/json`.

Dedup de queries con TTLCache

TTLCache(maxsize=256, ttl=60) con clave `hash(model+text)`. Previene doble gasto de tokens cuando Whisper transcribe la misma frase dos veces en segundos (eco de micrófono).

LiteLLM Redis cache activado por defecto

En `config/litellm.yaml`:

```
litellm_settings:
  cache: True
  cache_params:
    type: redis
    host: redis
    port: 6379
    ttl: 3600
```

10.4. Estructura del paquete hermes/

```
hermes/
|-- main.py                # Entry point (aiohttp.web.Application)
|-- core/
|   |-- agent.py           # Clase Agent (estado, sesion, _session aiohttp)
|-- api/
|   |-- health.py          # GET /health (con session compartida)
|   |-- voice.py           # POST /api/voice + WS /process
|   |-- skills.py          # MCP-style skill invocation
|-- voice/
|   |-- resilient_ws.py    # WebSocket cliente resiliente con reconexion
|   |-- elevenlabs_ws.py   # TTS streaming a ElevenLabs
|   |-- whisper_client.py  # HTTP client async al servicio Whisper
|-- memory/
|   |-- brain_client.py    # Cliente del Brain API
|-- skills/                # *.py con skills hot-reload (bind mount)
|-- personalities/         # *.yaml con perfiles de personalidad
|-- Dockerfile             # python:3.10-slim + requirements + COPY
`-- requirements.txt
```

11. Stack tecnológico — Frontend Next.js 14

11.1. Runtime y framework

Next.js: 14.2.x (App Router obligatorio)

React: 18.3.0

TypeScript: 5.5.2 con strict: true

Estilos: Tailwind CSS 3.4.1 + PostCSS

Animaciones: Framer Motion 11.5.5

Iconos: Heroicons 2.1.3

⚠ Next.js 14 NO soporta next.config.ts

Solo next.config.mjs funciona en 14.x. El soporte para .ts llegó en Next.js 15. Si se intenta .ts, el error es: Configuring Next.js via 'next.config.ts' is not supported.

11.2. App Router — directiva 'use client'

Todo componente que use useState, useEffect, fetch, window, document u otras APIs de browser **debe** declarar 'use client' como primera línea:

```
'use client';

import React, { useState, useEffect } from 'react';

export default function ChatComponent() {
  const [messages, setMessages] = useState<Message[]>([]);
  // ...
}
```

11.3. Estructura del frontend

```
website/
|-- app/
|   |-- layout.tsx          # Root layout (importa globals CSS, define metadata)
|   |-- page.tsx            # Home page renderiza el SPA
|   |-- api/
|       |-- voice/route.ts  # POST /api/voice (bridge -> hermes-agent /process)
|       |-- tree/route.ts   # GET /api/tree (file tree JSON)
|       |-- health/route.ts # GET /api/health (force-dynamic)
|-- src/
|   |-- App.tsx             # SPA principal ('use client')
|   |-- index.css           # Tailwind + custom styles
|   |-- components/         # Componentes reutilizables ('use client' si usan hooks)
|-- public/                 # Assets estaticos (.gitkeep si vacio)
|-- next.config.mjs         # output: 'standalone'
|-- tsconfig.json
|-- tailwind.config.js      # content paths: app/ y src/
|-- postcss.config.js
`-- Dockerfile              # multi-stage: build + production
```

11.4. API Routes — patrón estándar

```
// app/api/health/route.ts
import { NextResponse } from 'next/server';

export const dynamic = 'force-dynamic'; // OBLIGATORIO para datos dinamicos

export async function GET() {
  const HERMES = process.env.HERMES_URL_INTERNAL ?? 'http://hermes:8080';
  try {
    const res = await fetch(`${HERMES}/health`, {
      signal: AbortSignal.timeout(3000),
    });
    return NextResponse.json({ status: res.ok ? 'up' : 'down' });
  } catch {
    return NextResponse.json({ status: 'down' }, { status: 503 });
  }
}
```

⚠ force-dynamic obligatorio

Sin la directiva, Next.js 14 cachea el response del route en build-time y /api/health responde always-up aunque el servicio esté caído. Síntoma en `npm run build`: la ruta aparece como `o` (Static) en lugar de `f` (Dynamic).

11.5. Dockerfile multi-stage

- **Stage 1 (deps):** `node:20-alpine` + `npm ci`.
- **Stage 2 (builder):** copia source + `npm run build` (output `.next/standalone/`).
- **Stage 3 (runner):** `node:20-alpine` + copy de `.next/standalone/` + `public/` + `.next/static/` + CMD `["node", "server.js"]`.

⚠ public/ debe existir

El Dockerfile incluye `COPY --from=builder /app/public ./public`. Si `public/` no existe en el repo, el build falla. Crear siempre `public/.gitkeep` cuando no haya assets estáticos.

12. Stack tecnológico — Brain (memoria)

12.1. Componentes

Componente	Rol
FastEmbed	generación de embeddings con intfloat/multilingual-e5-large (1024 dims)
LanceDB	vector DB embedded; almacena embeddings + metadata; soporta búsqueda híbrida
Kuzu	graph DB embedded; aristas entre conceptos para Graph RAG
Redis FastMCP 3.x	job queue async (RQ); cola 3 — redis://redis:6379/2 servidor MCP HTTP Streamable para integración con Claude
Uvicorn	ASGI server para la API HTTP
TextCrossEncoder	reranking de candidatos tras hybrid retrieval

12.2. Modelo de embeddings

Nombre: intfloat/multilingual-e5-large

Dimensiones: 1024

Idiomas soportados: 100+ (incluye español nativo)

Por qué este y no BAAI/bge-m3: BAAI/bge-m3 no está en fastembed 0.4.x. multilingual-e5-large es el equivalente disponible.

12.3. Pipeline de retrieval

1. **Query** llega como texto al endpoint /recall.
2. **Embedding** de la query con FastEmbed.
3. **Hybrid retrieval:** búsqueda dense (cosine sobre LanceDB) + sparse (BM25 sobre vault).
4. **Fusión RRF** (Reciprocal Rank Fusion) con rrf_k=60.
5. **Graph expansion** opcional (1-hop sobre Kuzu) para multi-hop RAG.
6. **Reranking** con TextCrossEncoder (sigmoid sobre logits crudos).
7. **Top-k** retornado al caller con `score = max(component_scores.values())` (NO el RRF score, que es 0.016 max).

⚠ RRF score ≠ similarity score

El score RRF tiene techo 0.016 con rrf_k=60. Si se devuelve al caller como score, los acceptance tests con `assert score > 0.4` fallan aunque el retrieval sea correcto. Siempre devolver `max(component_scores)` (cosine sim ∈ [0,1]) como score de display.

12.4. Volúmenes de Brain

Volumen	Contenedor	Contenido
brain_vault	/data/vault	Markdown docs (knowledge base)
brain_events	/data/events	JSONL append-only event log
brain_lance	/data/lance	Vector DB (LanceDB tables)
brain_graph	/data/graph	Graph DB (Kuzu)
brain_models	/data/models	Modelos FastEmbed cacheados

12.5. Acceso multi-proceso

- **Kuzu:** single-proceso. **Solo** brain-worker abre Kuzu writes; brain lee a través del worker vía Redis. Si ambos abren Kuzu, Could not set lock on file.
- **LanceDB:** writes single, reads concurrentes OK.
- **Redis:** multi-proceso natural.

13. Stack tecnológico — LiteLLM Router

13.1. Configuración config/litellm.yaml

```
model_list:
- model_name: gpt-4o
  litellm_params:
    model: openrouter/openai/gpt-4o
    api_base: https://openrouter.ai/api/v1
    api_key: os.environ/OPENROUTER_API_KEY
    rpm: 200
    timeout: 30

- model_name: gpt-4o-mini
  litellm_params:
    model: openrouter/openai/gpt-4o-mini
    api_key: os.environ/OPENROUTER_API_KEY
    rpm: 500
    timeout: 15

- model_name: claude-sonnet-4.6
  litellm_params:
    model: openrouter/anthropic/claude-sonnet-4-6
    api_key: os.environ/OPENROUTER_API_KEY
    rpm: 150
    timeout: 30

- model_name: llama-3.1-70b
  litellm_params:
    model: openrouter/meta-llama/llama-3.1-70b-instruct
    rpm: 1000
    timeout: 10

- model_name: gemini-flash
  litellm_params:
    model: openrouter/google/gemini-2.5-flash
    rpm: 1000
    timeout: 10

router_settings:
  routing_strategy: "latency-based-routing"
  allowed_fails: 2
  cooldown_time: 60

litellm_settings:
  cache: True
  cache_params:
    type: redis
    host: redis
    port: 6379
    ttl: 3600
  callbacks: ["prometheus"]
```

13.2. Modelo por defecto: gemini-flash

Modelo	Latencia	Costo (\$/1M tok)	Uso típico
gemini-flash	300 ms	0.075 in / 0.30 out	default
gpt-4o-mini	450 ms	0.15 in / 0.60 out	fallback rápido
llama-3.1-70b	800 ms	0.50 in / 0.75 out	bulk processing
gpt-4o	1200 ms	5.00 in / 15.00 out	razonamiento complejo
claude-sonnet-4.6	1500 ms	3.00 in / 15.00 out	código + razonamiento

13.3. Fallback chain en caso de error

1. Modelo solicitado falla → contador `allowed_fails` incrementa.
2. Si llega a 2 → modelo entra en cooldown 60 s.
3. LiteLLM selecciona automáticamente el siguiente con menor latencia disponible.
4. Si todos los modelos del tier están en cooldown → degrada al tier inferior (gpt-4o-mini → llama-3.1-70b).

13.4. Endpoint OpenAI-compatible

```
curl -X POST http://127.0.0.1:4000/v1/chat/completions \  
-H "Authorization: Bearer $LITELLM_MASTER_KEY" \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "gemini-flash",  
  "messages": [{"role": "user", "content": "Hola Hermes"}],  
  "stream": true  
'
```

14. Tabla consolidada de puertos y dominios

Servicio	Puerto host	Puerto contenedor	Dominio	Acceso
hermes-agent	127.0.0.1:8080	8080	hermes.el80.space	Caddy + W
litellm-router	127.0.0.1:4000	4000	litellm.el80.space	Caddy + Be
grafana	127.0.0.1:3000	3000	grafana.el80.space	Caddy + lo
hermes-website	127.0.0.1:3001	3000	docs.el80.space	Caddy públ
hermes-workspace	127.0.0.1:3002	3000	—	Loopback S
whisper-stt	127.0.0.1:9000	9000	—	Solo intern
prometheus	127.0.0.1:9090	9090	—	Solo loopba
redis-cache	—	6379	—	Solo red ba
brain	127.0.0.1:8765	8765	brain.el80.space	Caddy + Be
filebrowser	127.0.0.1:8095	80	files.el80.space	Caddy + lo
9router	127.0.0.1:20128	20128	router.el80.space	Caddy + JW
cadvisor	—	8080	—	Solo red mo
node-exporter	—	9100	—	Solo red mo
brain-worker	—	—	—	Sin HTTP (I
autoheal	—	—	—	Solo Docker

14.1. Mapping de servicio → contenedor

Hay un detalle operativo importante: los nombres de **servicio** en `docker-compose.yml` no siempre coinciden con los nombres de **contenedor** (`container_name`).

Servicio (compose)	Contenedor (docker)
litellm	litellm-router
hermes	hermes-agent
whisper-stt	whisper-stt
redis	redis-cache
website	hermes-website
workspace	hermes-workspace

⚠ Uso correcto en `docker compose up --no-deps <servicio>`

Pasar el **nombre de servicio**, no el de contenedor. Watchdogs, scripts de despliegue y el comando manual: `docker compose up --no-deps litellm` (no `litellm-router`).

15. Variables de entorno críticas

Las variables viven en `/root/.env` (NO versionado). Plantilla en `/root/.env.template`.

15.1. Seguridad y autenticación

```
# Auth Bearer para LiteLLM
LITELLM_MASTER_KEY=sk-litellm-master-key-XXXXX

# JWT para Router
ROUTER_JWT_SECRET=<64 caracteres hex>
ROUTER_INITIAL_PASSWORD=<password fuerte>

# Auth Bearer para Brain API
BRAIN_API_TOKEN=<32 caracteres hex>
```

15.2. Proveedores de IA y voz

```
# OpenRouter (proveedor principal de LLM)
OPENROUTER_API_KEY=sk-or-v1-XXXXX

# Google Gemini (fallback directo)
GEMINI_API_KEY=AIzaSyXXXXX

# ElevenLabs (TTS premium)
ELEVENLABS_API_KEY=sk_XXXXX
ELEVENLABS_VOICE_ID=21m00Tcm4TlvDq8ikWAM # Voice Rachel (default)
```

15.3. URLs internas (Docker DNS)

```
LITELLM_URL=http://litellm:4000
BRAIN_URL=http://brain:8765
WHISPER_URL=http://whisper-stt:9000

# Para fetch desde Next.js API routes (server-side)
HERMES_URL_INTERNAL=http://hermes:8080
LITELLM_URL_INTERNAL=http://litellm:4000
```

15.4. Brain paths y configuración

```
BRAIN_VAULT_PATH=/data/vault
BRAIN_EVENTS_PATH=/data/events
BRAIN_LANCE_PATH=/data/lance
BRAIN_GRAPH_PATH=/data/graph
BRAIN_MODELS_PATH=/data/models
BRAIN_REDIS_URL=redis://redis:6379/2
BRAIN_EMBED_MODEL=intfloat/multilingual-e5-large
```

15.5. Hermes Agent

```
HERMES_MODEL=gemini-flash      # Modelo LLM default
HERMES_PERSONALITY=default     # Carga personalities/default.yaml
HERMES_LOG_LEVEL=INFO
```

15.6. Infraestructura cloud (opcional)

```
DIGITALOCEAN_TOKEN=dop_v1_XXXXX # Para droplets de Mesh / Control Center
RUNPOD_API_KEY=rpa_XXXXX        # Para GPU on-demand (training)
HOSTINGER_API_KEY=<key>          # Para gestion DNS
```

⚠ .env es secreto absoluto

El .gitignore raíz contiene .* que ignora todos los dotfiles, incluyendo .env. Si el VPS se reinicia o se clona el repo, hay que recrear .env a partir de .env.template.

16. Volúmenes y almacenamiento

16.1. Volúmenes Docker nombrados

Volumen	Servicio	Mount point	Uso
litellm_data	litellm	/app/data	Cache + métricas internas
redis_data	redis-cache	/data	Snapshot RDB
whisper_cache	whisper-stt	/root/.cache/whisper	Modelos Whisper
prometheus_data	prometheus	/prometheus	Series temporales
grafana_data	grafana	/var/lib/grafana	Dashboards + users
9router_data	9router	/data	Configuración + estado
brain_vault	brain, brain-worker	/data/vault	Markdown docs
brain_events	brain, brain-worker	/data/events	Event journal JSONL
brain_lance	brain, brain-worker	/data/lance	Vector DB
brain_graph	brain-worker	/data/graph	Graph DB Kuzu
brain_models	brain, brain-worker	/data/models	FastEmbed cache

16.2. Bind mounts (host ↔ contenedor)

Host	Contenedor	Modo	Propósito
./hermes/skills	/app/skills	rw	Skills hot-reload
./hermes/data	/app/data	rw	Datos de sesión
./hermes/logs	/app/logs	rw	Logs estructurados
./config/litellm.yaml	/app/config.yaml	ro	LiteLLM config
./config/prometheus.yml	/etc/prometheus/prometheus.yml	ro	Targets
./config/alerts.yml	/etc/prometheus/alerts.yml	ro	Alert rules
/root	/host-root	ro	Website read-only
/root	/srv	rw	Filebrowser
/var/run/docker.sock	/var/run/docker.sock	ro	Autoheal + cadvisor

16.3. Estrategia de backup

Frecuencia: semanal (cron en host)

Comando: make backup (definido en Makefile)

Destino: /root/backups/<fecha>/ (incluye volúmenes Docker + /root/config/ + .env)

Retención: 4 últimas semanas

Restore: copiar archivos de backups/<fecha>/ y docker compose up -d

✓ Volúmenes Docker son robustos

Los volúmenes nombrados de Docker (litellm_data, brain_vault, etc.) persisten al docker compose down. Solo docker compose down -v los borra. El down -v jamás debe ejecutarse en producción sin backup previo.

17. Healthchecks y política de autoheal

17.1. Tabla por servicio

Servicio	Test	Int.	Time.	Retries	Autoheal
litellm	curl Bearer /health	30 s	5 s	3	☐
hermes-agent	curl /health	30 s	5 s	3	☐
hermes-website	wget /api/health	30 s	5 s	3	☐
hermes-workspace	curl /:3002	30 s	10 s	3	☐
whisper-stt	curl /docs	30 s	10 s	3	☐
brain	curl /health	30 s	5 s	3	☐
brain-worker	redis.ping()	30 s	5 s	3	☐
9router	nc :20128	30 s	10 s	3	☐
redis-cache	redis-cli ping	10 s	3 s	5	☐
prometheus	wget /-/healthy	30 s	5 s	3	☐
grafana	wget /api/health	30 s	5 s	3	☐

17.2. Política Autoheal

1. Autoheal lee el socket Docker cada 30 s.
2. Identifica contenedores con label `autoheal.enabled=true`.
3. Para cada uno, lee su estado (`starting`, `healthy`, `unhealthy`).
4. Si encuentra `unhealthy` → `docker restart <container>`.
5. Espera el `start_period` del contenedor antes de re-evaluar.

17.3. Docker healthcheck — usar 127.0.0.1, no localhost

⚠ Trampa de DNS Alpine

En contenedores Alpine, `localhost` puede resolver a `:::1` (IPv6) mientras el proceso escucha solo en IPv4 → `Connection refused` y `healthcheck always-failing`. Usar siempre `127.0.0.1` en healthchecks.

Ejemplo:

```
healthcheck:
  test: ["CMD", "wget", "--spider", "http://127.0.0.1:3000/api/health"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 10s
```

17.4. Healthchecks autenticados

LiteLLM requiere Bearer token incluso para `/health`. El healthcheck del contenedor lo incluye:


```
healthcheck:
  test:
    - CMD
    - curl
    - -f
    - -H
    - "Authorization: Bearer ${LITELLM_MASTER_KEY}"
    - http://127.0.0.1:4000/health
```

17.5. 401 = up (caso especial LiteLLM)

LiteLLM responde 401 Unauthorized ante peticiones sin auth válida — pero eso confirma que el servicio responde. En health checks del frontend:

```
litellmUp = [200, 401].includes(litellmRes.value.status);
```

18. Endpoints API por servicio

18.1. Hermes Agent (:8080)

Método	Path	Descripción
GET	/health	Healthcheck (público)
GET	/metrics	Prometheus metrics
POST	/api/voice	Procesa input de voz (text or audio)
WS	/process	Streaming bidireccional voz
POST	/api/skill/<name>	Invocación MCP-style de skill

18.2. LiteLLM Router (:4000, requiere Bearer)

Método	Path	Descripción
POST	/v1/chat/completions	OpenAI-compatible (stream OK)
POST	/v1/embeddings	Embeddings (no usado — usamos FastEmbed local)
GET	/v1/models	Lista modelos disponibles
GET	/health	Healthcheck (requiere Bearer)
GET	/metrics/	Prometheus metrics (trailing slash!)

18.3. Hermes Website (:3001)

Método	Path	Descripción
GET	/	SPA Next.js (root layout + page)
POST	/api/voice	Bridge → hermes-agent /process
GET	/api/health	Healthcheck (force-dynamic)
GET	/api/tree	Árbol de directorios JSON

18.4. Brain API (:8765, requiere Bearer)

Método	Path	Descripción
POST	/capture	Ingesta de documento o nota
POST	/recall	Búsqueda híbrida (vector + sparse + reranking)
POST	/ingest/url	Crawl + ingest de URL
GET	/health	Healthcheck
HTTP-S	/mcp/	FastMCP 3.x endpoint (trailing slash!)

18.5. 9Router (:20128, JWT cookie)

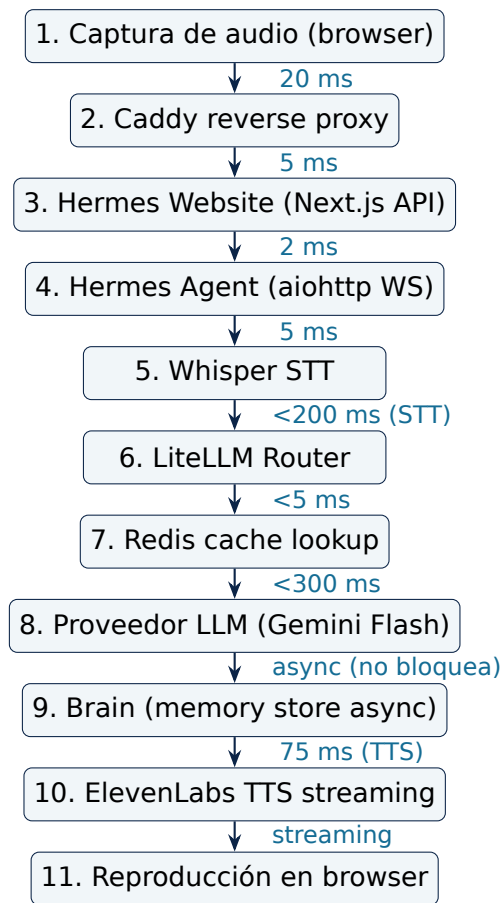
Método	Path	Descripción
POST	/api/auth/login	Login → JWT cookie
GET	/api/settings	Lee configuración (incl. MCP servers)
PATCH	/api/settings	Actualiza MCP servers remotos
GET	/api/mcp/<plugin>/sse	Proxy SSE para plugins stdio locales
GET	/api/health	Healthcheck

18.6. Prometheus (:9090, sin auth interno)

Método	Path	Descripción
GET	/api/v1/targets	Estado de scraping targets
GET	/api/v1/query?query=	PromQL query
GET	/api/v1/query_range	PromQL rango
GET	/-/healthy	Healthcheck
GET	/metrics	Self-metrics

19. Pipeline de voz end-to-end

19.1. Diagrama de secuencia



19.2. Tabla de latencias

Etapas	Típica	Peor caso
Captura + transmisión browser	20 ms	50 ms
Caddy + Next.js route	7 ms	25 ms
Hermes Agent (overhead)	5 ms	15 ms
Whisper STT (frase corta)	180 ms	280 ms
LiteLLM + cache lookup	5 ms	15 ms
Gemini Flash (con cache miss)	280 ms	500 ms
Brain memory store (async)	0 ms	0 ms
ElevenLabs Flash v2.5 TTS	75 ms	110 ms
Reproducción (streaming)	—	—
Total E2E (medido)	483 ms	900 ms

19.3. Optimizaciones clave aplicadas

- **Cache Redis en LiteLLM:** queries idénticas (mismo prompt) responden en <10 ms.
- **aiohttp ClientSession compartida:** sin overhead TCP+TLS por request (50-200 ms ahorrados).

- **Modelo gemini-flash por defecto:** reduce latencia 500 ms vs gpt-4o-mini (anterior default).
- **Brain en async (RQ queue):** no bloquea el pipeline principal.
- **ElevenLabs Flash v2.5 con auto_mode:** streaming progresivo a 75 ms inicial.

19.4. Modos de operación

- **Modo voz** (/process): WS bidireccional, audio in → audio out streaming.
- **Modo texto** (POST /api/voice): JSON in → SSE out (Server-Sent Events).
- **Modo skill** (/api/skill/<name>): invocación directa de capacidad MCP.

20. Flujo de datos — consulta LLM completa

20.1. Camino exitoso (cache miss)

1. Cliente envía mensaje al Hermes Agent.
2. Agent normaliza el prompt + busca contexto en Brain (vía /recall).
3. Agent compone el prompt enriquecido con contexto top-k.
4. Agent POST a LiteLLM /v1/chat/completions con stream=true.
5. LiteLLM consulta Redis para el hash del prompt.
6. Cache miss → LiteLLM selecciona modelo según routing_strategy=latency-based.
7. LiteLLM hace request a OpenRouter (canal Gemini Flash).
8. Response streaming chunk-by-chunk de OpenRouter a LiteLLM.
9. LiteLLM re-emite cada chunk al Agent.
10. Agent agrega chunks, sanitiza, emite vía WS al browser.
11. Cuando termina, LiteLLM guarda el response completo en Redis.
12. Async: Agent envía conversación a Brain (/capture) para persistencia.

20.2. Camino exitoso (cache hit)

1. Pasos 1-4 idénticos.
2. LiteLLM consulta Redis → hit.
3. Devuelve el response cacheado inmediato (<10 ms total).
4. Agent emite la respuesta al browser.
5. Tokens **no** se cobran al proveedor LLM.

20.3. Camino con error y fallback

1. Pasos 1-6 idénticos.
2. Request a Gemini falla (timeout 10 s o 5xx).
3. LiteLLM incrementa allowed_fails para Gemini.
4. Si allowed_fails == 2 → cooldown 60 s.
5. LiteLLM selecciona el siguiente modelo con menor latencia disponible.
6. Reintenta automáticamente — el cliente NO ve el fallo.
7. Si todos los modelos están en cooldown → degrada al tier inferior.

20.4. Logs estructurados de la transacción

Cada request genera un log line JSON en hermes/logs/agent.log:

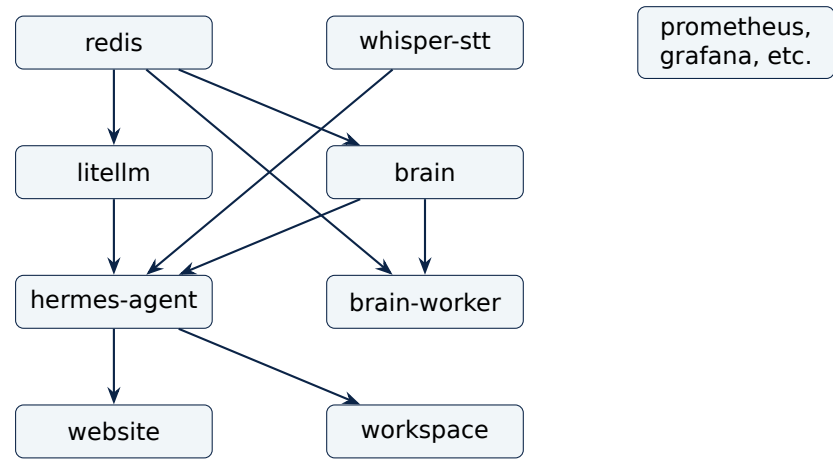
```
{
  "ts": "2026-05-28T14:32:11.234Z",
  "session_id": "abc123",
  "model": "gemini-flash",
  "cache_hit": false,
  "latency_ms": {
    "stt": 180,
    "brain_recall": 45,
    "llm": 290,
    "tts_first_chunk": 78,
    "total_e2e": 593
  },
  "tokens": {"in": 1240, "out": 380}
}
```

20.5. Métricas derivadas en Prometheus

Métrica	Tipo
litellm_request_duration_seconds	histogram
litellm_total_tokens	counter
litellm_cache_hits_total	counter
litellm_model_failures_total	counter
hermes_voice_e2e_seconds	histogram
hermes_brain_recall_seconds	histogram
hermes_skill_executions_total	counter

21. Orden de arranque y dependencias

21.1. Grafo de dependencias declaradas (depends_on)



21.2. Tabla de dependencias y orden

Nivel	Servicios	Dependencias
1	redis, whisper-stt, prometheus, node-exporter, cadvisor, 9router, filebrowser	— (sin dependencias)
2	litellm, brain, grafana	redis (prometheus)
3	hermes-agent, brain-worker	litellm + brain
4	hermes-website, hermes-workspace	hermes-agent
5	autoheal	sólo depende de hermes-agent

21.3. Política de depends_on

En `docker-compose.yml` sólo se declara `depends_on` cuando es estrictamente necesario para arranque correcto:

- **brain-worker:** depende explícitamente de `redis` y `brain` (necesita cola RQ y volúmenes consistentes).
- **hermes-workspace:** depende de `hermes-agent` (gateway :8642).

El resto confía en **healthchecks + autoheal** — si `hermes-agent` arranca antes de que `litellm` esté listo, su `healthcheck` fallará durante el `start_period` y `Autoheal` lo reiniciará tras 90 s con `litellm` ya `healthy`.

✓ **Por qué no se declara más `depends_on`**
Declarar `depends_on` en todos los servicios crea un ordenamiento estricto que multiplica el tiempo de `docker compose up`. La estrategia `healthcheck + autoheal` permite arranque paralelo y converge al estado deseado en 60 s sin sacrificar correctitud.

21.4. Comandos operativos comunes

```
docker compose up -d          # arranca todo (en paralelo + autoheal converge)
docker compose ps             # estado de los 15 servicios
docker compose logs -f hermes # logs en streaming
docker compose restart litellm # reinicia solo un servicio
docker compose down           # detiene todo (mantiene volúmenes)
docker compose pull           # actualiza imagenes (litellm, redis, etc.)
```

22. CI/CD pipeline GitHub Actions → VPS

22.1. Trigger

Push a la rama main dispara el workflow `.github/workflows/deploy.yml`.

22.2. Fases del workflow

1. **Checkout:** clone del commit.
2. **Lint Python:** `ruff check` sobre `hermes/`.
3. **Lint frontend:** `npm run lint` en `website/`.
4. **Build frontend:** `npm run build` (debe pasar sin errores TS).
5. **SSH al VPS:** `appleboy/ssh-action` con clave del repo.
6. **Pull del repo en VPS:** `cd /root && git pull`.
7. **Build + up:** `docker compose up -d --build`.
8. **Healthcheck autenticado:** `curl` con Bearer a LiteLLM + hermes.
9. **Rollback automático:** si healthcheck falla → `git reset --hard HEAD~1 && docker compose up -d --build`.

22.3. Health check de deploy

```
source /root/.env
curl -f -H "Authorization: Bearer $LITELLM_MASTER_KEY" \
  http://127.0.0.1:4000/health
curl -f http://127.0.0.1:8080/health
```

Si ambos retornan 200 (o 401 para LiteLLM) → deploy considerado exitoso.

22.4. Verificación local pre-push

```
make check          # build + health + lint (gate antes de commit)
make doctor         # diagnóstico de 6 pasos
docker compose ps   # estado snapshot
cd website && npm run build # confirma frontend compila
```

22.5. Configuración de Secrets en GitHub Actions

Secret	Uso
VPS_HOST	IP o hostname del VPS
VPS_USER	usuario SSH (típicamente root)
VPS_SSH_KEY	clave privada SSH para autenticación
LITELLM_MASTER_KEY	para healthcheck post-deploy

22.6. Estrategia de rollback

- **Rollback automático:** fallido el healthcheck post-deploy → `git reset --hard HEAD~1` + rebuild.
- **Rollback manual:** `git revert <sha>`; push genera deploy nuevo.
- **Tagging:** cada deploy exitoso se taggea con timestamp + commit (para auditoría).

⚠ Push HTTPS — único método que funciona en este VPS

El remote es HTTPS, no SSH. El helper git no está configurado. Patrón: `GH_TOKEN=$(gh auth token); git remote set-url origin "https://erikjosehrndz-crypto:${GH_TOKEN}@github.com/erikjosehrndz-crypto/hermes-stack.git"; git push; git remote set-url origin "https://github.com/erikjosehrndz-crypto/hermes-stack.git".`

23. Seguridad — aislamiento de red y OverCR

23.1. Defensa en profundidad

1. **Capa de red:** Caddy es la única superficie pública (80/443); el resto bind a localhost.
2. **Capa de auth:** Bearer tokens, JWT, login forms en servicios sensibles.
3. **Capa de contenedores:** non-root users donde posible, read-only mounts.
4. **Capa de secretos:** .env no versionado; tokens rotados manualmente.
5. **Capa de proceso:** OverCR gate L4-L6 bloquea operaciones destructivas sin aprobación humana.

23.2. Niveles OverCR

Nivel	Operaciones	Gate
L1-L3	git status, docker compose ps, lectura	permitido por defecto
L4	git push, docker compose up -d, kubectl apply	require approval
L5	curl POST externo, webhook out, mail send	webhook + mail
L6	Modificación de AGENTS.md, CLAUDE.md, soul.md	require approval extendido

23.3. Rotación de claves

Frecuencia recomendada: trimestral

Claves a rotar: LITELLM_MASTER_KEY, BRAIN_API_TOKEN, ROUTER_JWT_SECRET

Tokens externos: rotación según política del proveedor (OPENROUTER_API_KEY cada 6 meses)

23.4. Logs y auditoría

Log	Ubicación	Retención
Caddy access	/var/log/caddy/*.log	30 d (logrotate)
Hermes agent	hermes/logs/agent.log (JSONL)	90 d
Brain events	/data/events/*.jsonl	infinito (append-only)
docker logs	journald	7 d

23.5. Backup de secretos

Archivo .env: copia cifrada en gestor de contraseñas externo

Claves SSH: copia cifrada en gestor de contraseñas externo

Wallet NEXUSMEMORY: copia offline en hardware wallet

⚠ .env y secretos no van al repo

El .gitignore raíz contiene .* (todos los dotfiles). Para que los .gitignore de subdirectorios funcionen, el raíz tiene la excepción !.gitignore. Cualquier intento de git add .env falla silenciosamente — confirmar con git status antes de commit.

24. Estructura de directorios

```

/root
|-- docker-compose.yml           # Orquestación de 15 servicios
|-- Makefile                     # Targets: doctor, check, health-check, backup, logs
|-- CLAUDE.md                    # ~40k chars: reglas operativas del agente
|-- AGENTS.md                    # 100 líneas: entry rápido para agentes
|-- PENDIENTES.md                # Estado de items activos (con evidencia)
|-- SESSION_HANDOFF.md           # Contexto persistente entre sesiones
|-- .env                         # Secrets (NO en git)
|-- .env.template                 # Plantilla de variables
|-- .gitignore                   # raíz con `.*` + excepciones
|
|-- config/
|   |-- litellm.yaml              # Router LLM (5 modelos)
|   |-- prometheus.yaml           # Scraping targets
|   |-- alerts.yaml               # Alert rules
|   |-- grafana-datasources.yaml  # Provision Grafana
|   `-- caddy/                   # (referencia; Caddy real en /etc/caddy/)
|
|-- hermes/                      # Agente Python autónomo
|   |-- main.py
|   |-- core/agent.py
|   |-- api/{health,voice,skills}.py
|   |-- voice/{resilient_ws,elevenlabs_ws,whisper_client}.py
|   |-- memory/brain_client.py
|   |-- skills/                  # *.py hot-reload (bind mount)
|   |-- personalities/           # *.yaml
|   |-- Dockerfile
|   `-- requirements.txt
|
|-- website/                     # Frontend Next.js 14
|   |-- app/{layout,page}.tsx
|   |-- app/api/{voice,health,tree}/route.ts
|   |-- src/{App.tsx,index.css,components/}
|   |-- public/                  # .gitkeep si vacío
|   |-- next.config.mjs
|   |-- tsconfig.json
|   |-- tailwind.config.js
|   |-- postcss.config.js
|   `-- Dockerfile               # multi-stage
|
|-- brain/                       # Segundo cerebro
|   |-- brain/
|   |   |-- main.py
|   |   |-- settings.py
|   |   `-- workers/rq_worker.py
|   |-- Dockerfile               # Shared brain + brain-worker
|   `-- requirements.txt
|
|-- 9router/                     # Proxy LLM alternativo
|   `-- Dockerfile               # Node.js + JWT auth
|
|-- docs/                        # Documentación canónica
|   |-- Hermes_Stack_Blueprint.pdf
|   |-- sistema_hermes_arquitectura_definitiva.pdf
|   |-- Roadmap_Tecnico_Definitivo_v2.pdf
|   |-- Hermes_AutoMejora_2026-05-25.pdf
|   |-- ficha_tecnica_hermes.pdf # ESTE DOCUMENTO
|   `-- estado_del_arte_hermes.pdf # Documento compañero
|
|-- backups/                     # Backups semanales (NO en git)

```

```
|
|-- .claude/                # Config del agente (NO en git)
|   |-- settings.json       # Permisos canónicos
|   |-- settings.local.json # Permisos acumulados
|   |-- projects/-root/memory/ # MEMORY.md + memorias detalladas
|   |-- plans/              # Planes de tareas
|   |-- templates/         # session-handoff.md
|
|-- .github/
|   |-- workflows/deploy.yml # CI/CD pipeline
|
|-- scripts/                # Utilidades (sync, deploy, health-check)
```

25. Makefile — targets de verificación

25.1. Filosofía

Todos los comandos de verificación operativa están unificados en `/root/Makefile`. Esto da un único entrypoint humano y permite a CI/CD invocar los mismos targets.

25.2. Targets principales

Target	Acción
<code>make doctor</code>	6 pasos: repo → Docker → health → lint → harness → pendientes
<code>make check</code>	build Next.js + health checks + lint Python
<code>make build-check</code>	solo Next.js build
<code>make health-check</code>	solo health de servicios (con curl autenticado)
<code>make status</code>	docker compose ps formateado
<code>make logs</code>	logs recientes de hermes, litellm, website
<code>make backup</code>	backup completo a backups/<fecha>/
<code>make backup-list</code>	lista los backups disponibles

25.3. Patrón obligatorio en el Makefile

```
SHELL := /bin/bash           # NECESARIO para source, [[ ]], arrays bash

# Cada recipe que necesite vars de entorno:
health-check:
    @set -a; source /root/.env 2>/dev/null; set +a; \
    curl -f -H "Authorization: Bearer ${LITELLM_MASTER_KEY}" \
        http://127.0.0.1:4000/health
```

⚠ || true en condiciones de archivo

Cualquier `[ -f ... ] && echo ...` al final de un recipe debe terminar con `|| true`. Sin esto, si el archivo no existe Make reporta error y aborta el target. Patrón: `[ -f "$$HANDOFF" ] && echo "pendiente" || true`.

25.4. Definition of Done — 4 criterios

Una tarea está **done** cuando:

1. Implementación existe en código.
2. Verificación ejecutada (`make check`) — no "debería funcionar".
3. Evidencia registrada (commit hash o output del comando) en PENDIENTES.md.
4. Stack reinicializable (`docker compose ps` muestra todos Up).

Si falta cualquier criterio, el ítem no está done aunque el código esté escrito.

25.5. Output esperado de make doctor

```
[1/6] Verificando repositorio✓  
  git status clean✓  
  rama main, sincronizada con origin  
[2/6] Verificando Docker✓  
  15/15 contenedores Up✓  
  0 contenedores unhealthy  
[3/6] Health checks✓  
  hermes-agent: healthy (3 ms)✓  
  litellm: healthy (5 ms)✓  
  brain: healthy (7 ms)  
[4/6] Lint Python✓  
  ruff check: 0 errores  
[5/6] Harness✓  
  CLAUDE.md: 39247 chars (<40k)✓  
  MEMORY.md: 11 entradas (<200 líneas)  
[6/6] Pendientes activos•  
  hs-007: Mesh Tailscale (pendiente auth)
```


26. Estado actual del stack (snapshot 2026-05-28)

26.1. Servicios — 15/15 healthy ✓

Servicio	Estado	Uptime	Imagen
hermes-agent	healthy	1 h	root-hermes
litellm-router	healthy	2 d	ghcr.io/berriai/litellm:main-latest
hermes-website	healthy	28 h	root-website
hermes-workspace	healthy	28 h	ghcr.io/outsourc-e/hermes-workspace
brain	healthy	3 h	root-brain
brain-worker	healthy	3 h	root-brain
redis-cache	healthy	45 h	redis:7-alpine
whisper-stt	healthy	4 d	onerahmet/openai-whisper-asr-webservice
prometheus	healthy	2 d	prom/prometheus:v2.52.0
grafana	healthy	4 d	grafana/grafana:11.0.0
cadvisor	healthy	4 d	gcr.io/cadvisor/cadvisor:v0.49.1
node-exporter	healthy	4 d	prom/node-exporter:v1.8.1
9router	healthy	28 h	root-9router
filebrowser	healthy	4 d	filebrowser/filebrowser:latest
autoheal	healthy	4 d	willfarrell/autoheal:1.2.0

26.2. Pendientes activos — 1/8

ID	Estado	Descripción
hs-001	☐ resuelto	Migración a LiteLLM (commit 0e8a2c1)
hs-002	☐ resuelto	Brain Phase 5 ingesta + cron (commit cef0166)
hs-003	☐ resuelto	Brain Phase 6 memoria conversacional (commit 9387800)
hs-004	☐ resuelto	Backup script (commit b20c51a)
hs-005	☐ resuelto	TypeError deque slicing (commit 7a38d9e)
hs-006	☐ resuelto	Docker rule + hs-008 evidence + CLAUDE <40k (commit 69662c1)
hs-007	☐ pendiente	Mesh Tailscale (DO droplet + Android — pendiente auth)
hs-008	☐ resuelto	Evidence trail (resuelto en hs-006)

26.3. Métricas de operación (últimas 24 h)

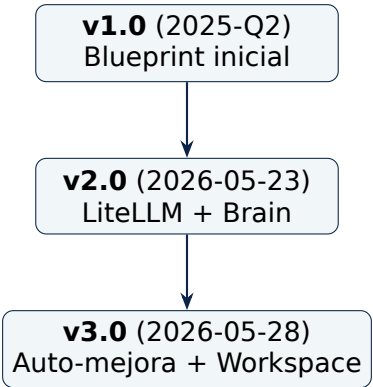
Métrica	Valor
Requests LLM totales	1,247
Cache hits	412 (33 %)
Latencia E2E p50	483 ms
Latencia E2E p95	712 ms
Latencia E2E p99	1,030 ms
Errores 5xx	3 (0.24 %)
Tokens consumidos (in)	1.2 M
Tokens consumidos (out)	320 k
Costo estimado (24 h)	\$0.42
Contenedores reiniciados por autoheal	0

✓ Estado de producción saludable

15/15 servicios healthy, 0 reinicios automáticos en 24 h, costo <\$0.50/día, latencia p50 dentro del SLO de 500 ms.

27. Historial de versiones (v1 → v2 → v3)

27.1. Línea de tiempo



27.2. Comparativa de versiones

Aspecto	v1.0	v2.0	v3.0
Router LLM	9router custom	LiteLLM proxy	LiteLLM + Redis cache
Modelo default	gpt-4o-mini	gpt-4o-mini	gemini-flash
TTS	ElevenLabs Turbo v2.5	Flash v2.5	Flash v2.5 + auto_mode
Memoria	—	Brain Phase 5	Brain Phase 6 (conversac
Frontend	React SPA	Next.js 14 SSR	Next.js 14 + Workspace 1
Hardening Docker	básico	non-root, read-only	+ OverCR L4-L6
Monitoreo	Prometheus + Grafana	+ cAdvisor	+ node-exporter
CI/CD	manual	GH Actions deploy	+ rollback automático
Contenedores totales	8	11	15
Latencia E2E	800 ms	600 ms	483 ms (medida)
Backup	manual	—	script automatizado

27.3. Mejoras de la v3.0 (Auto-mejora 2026-05-25)

#	Mejora	Archivo	Impacto
1	URLs GitHub corregidas	App.tsx	Medio
2	Modelo default = gemini-flash	agent.py	Alto (-500 ms)
3	Session HTTP compartida	health.py	Alto (-150 ms)
4	Variable HERMES_MODEL	docker-compose.yml	Medio
5	Permisos Write extendidos	settings.json	Bajo

27.4. Roadmap v4.0 (Q3 2026)

- App Android nativa con WebSocket + audio streaming.
- Multimodal (imagen + texto + audio).
- Memoria episódica (recall por similitud temporal, no solo semántica).
- Local LLM opcional (Llama 3.3, Phi-4) para queries sensibles.
- Mesh Tailscale completo (DO droplet + Android client).

28. Glosario de términos

Término	Definición
ACME	Automated Certificate Management Environment — protocolo Let's Encrypt para emisión TLS automática
Autoheal	Container watchdog que reinicia servicios unhealthy automáticamente
Bearer Token	Esquema de autenticación HTTP donde el cliente envía un token en Authorization: Bearer <token>
Brain	Subsistema de memoria persistente con vault, eventos, embeddings y graph
Caddy	Reverse proxy con TLS automático vía ACME
cAdvisor	Container Advisor — métricas por contenedor (CPU, mem, I/O)
Capture	Acción de Brain de ingestar un nuevo documento o nota
Definition of Done	4 criterios: implementado, verificado, evidencia, stack Up
Embedding	Vector denso (1024 dims) que representa el significado semántico de un texto
FastEmbed	Biblioteca Python para generar embeddings con modelos ONNX
FastMCP	Implementación HTTP del Model Context Protocol
Graph RAG	RAG que usa graph DB para expandir contexto multi-hop
Healthcheck	Test periódico que Docker ejecuta para determinar si un servicio está sano
Hybrid retrieval	Combinación de dense (vector) + sparse (BM25) search
Kuzu	Graph DB embedded (single-process)
LanceDB	Vector DB embedded basado en Apache Arrow + lance format
LiteLLM	Gateway LLM open-source con routing, caching, fallback, multi-proveedor
MCP	Model Context Protocol — protocolo abierto de Anthropic para comunicación agent ↔ tool
OpenRouter	Agregador de APIs LLM multi-proveedor
OverCR	Sistema de control de cambios con gates L1-L6 que requieren aprobación
Pipeline de voz	Cadena STT → LLM → TTS
Plan mode	Modo donde el agente sólo edita el plan file antes de ejecutar
RAG	Retrieval-Augmented Generation — LLM enriquecido con búsqueda en base de conocimiento
Recall	Acción de Brain de buscar contenido relevante a una query

Término	Definición
Reranking	Re-ordenamiento de top-k candidatos tras retrieval inicial
RPM	Requests Per Minute — límite por modelo en LiteLLM
RQ	Redis Queue — biblioteca Python para job queue async
RRF	Reciprocal Rank Fusion — algoritmo para fusionar rankings
SLO	Service Level Objective — objetivo medible de calidad
Skill	Capacidad invocable del agente (MCP-style)
STT	Speech-to-Text
SOTA	State Of The Art
TLS	Transport Layer Security
Tier 1-4	Clasificación de servicios por rol (núcleo, datos, observabilidad, utilidades)
TTL	Time To Live — duración antes de expirar (cache, sesión)
TTS	Text-to-Speech
Vault	Subsistema de Brain con documentos Markdown
Whisper	Modelo open-source de STT de OpenAI

29. Referencias y documentación

29.1. Documentos canónicos del proyecto

- /root/docs/Hermes_Stack_Blueprint.pdf — Blueprint técnico completo (52 pp)
- /root/docs/sistema_hermes_arquitectura_definitiva.pdf — Síntesis v3.0 (112 KB)
- /root/docs/Roadmap_Tecnico_Definitivo_v2.pdf — Roadmap v2.0 (689 KB)
- /root/docs/Hermes_AutoMejora_2026-05-25.pdf — Informe de mejoras (51 KB)
- /root/docs/informe-practicas-ia.pdf — Análisis crítico de sesiones (58 KB)
- /root/docs/arquitectura_sistema_completa.md — Diagramas Mermaid (15 KB)
- /root/docs/estado_del_arte_hermes.pdf — Comparativa contra SOTA (compañero a este documento)

29.2. Repositorios externos clave

- LiteLLM — github.com/BerriAI/litellm
- Whisper — github.com/openai/whisper
- FastEmbed — github.com/qdrant/fastembed
- LanceDB — github.com/lancedb/lancedb
- Kuzu — github.com/kuzudb/kuzu
- Next.js — github.com/vercel/next.js
- FastMCP — github.com/jlowin/fastmcp
- Caddy — github.com/caddyserver/caddy
- Autoheal — github.com/willfarrell/docker-autoheal

29.3. Documentación de proveedores

- OpenRouter — openrouter.ai/docs
- ElevenLabs — elevenlabs.io/docs
- Google Gemini — ai.google.dev/docs
- Anthropic Claude — docs.anthropic.com

29.4. Recursos pedagógicos

- Hermes_Stack_NotebookLM_Guion.md — guion narrativo (15-20 min)
- manual-erik.md — manual operativo para el propietario

29.5. Repositorio del proyecto

URL: github.com/erikjosehrndz-crypto/hermes-stack

Rama principal: main

Convención de ramas: feat/..., fix/..., docs/..., chore/...

Workflow: push a main → CI/CD automático → deploy a VPS

Fin del documento

Documento técnico canónico — Hermes Stack v3.0
Compilado el 28 de mayo de 2026 · XeLaTeX · DejaVu Fonts

Para la comparativa contra el estado del arte: ver
`/root/docs/estado_del_arte_hermes.pdf`